

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №5
по курсу «Программирование»
на тему «Определение характеристик графов»

Выполнили:

студенты группы 24ВВВ4:

Симаков А.С.

Орлов Р.А.

Принял:

к. т. н., доцент Юрова О.В.

к. э. н., доцент Акифьев И.В.

Пенза 2025

Цель работы:

Сгенерировать матрицу смежности для неориентированных графов G , вывести матрицу на экран, а также определить размер графа G , используя матрицу смежности графа и найти изолированные, концевые и доминирующие вершины.

Общие сведения.

Если G – граф (рисунок 1), содержащий непустое множество n вершин V и множество ребер E , где $e(v_i, v_j)$ – ребро между двумя произвольными вершинами v_i и v_j , тогда **размер** графа G есть мощность множества ребер $|E(G)|$ или, количество ребер графа.

Степенью вершины графа G называется число инцидентных ей ребер. Степень вершины v_i обозначается через $deg(v_i)$.

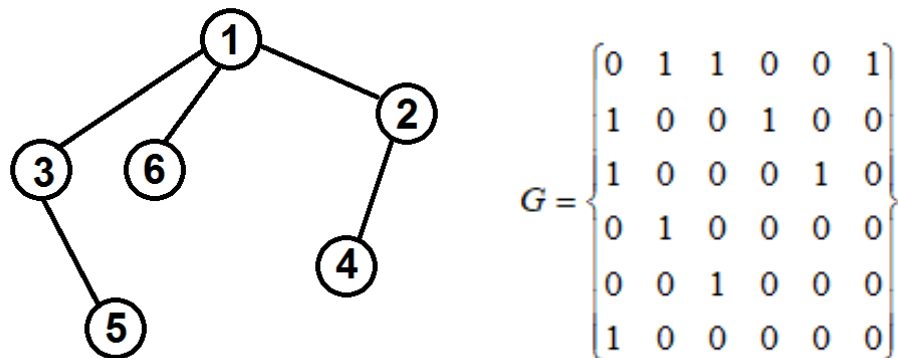


Рисунок 1 – Граф

Вершина v_i со степенью 0 называется **изолированной**, со степенью 1 – **концевой**.

Вершина графа, смежная с каждой другой его вершиной, называется **доминирующей**

Практическая часть

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Листинг программы

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <conio.h>

int main() {
    int** G;
    int* loop;
    int* deg;
    int N, size;
    int i, j;
    setlocale(LC_ALL, "rus");

    srand(time(NULL));

    printf("Введите количество вершин графа: ");
    scanf("%d", &N);

    G = (int**)malloc(sizeof(int*) * N);
    for(i = 0; i < N; i++) {
        G[i] = (int*)malloc(N * sizeof(int));
    }
    loop = (int*)malloc(N * sizeof(int));
    deg = (int*)malloc(N * sizeof(int));

    for(i = 0; i < N; i++) {
        deg[i] = 0;
        loop[i] = 0;
    }

    for(i = 0; i < N; i++) {
        for(j = i; j < N; j++) {
            {
                int value = rand() % 2;
                G[i][j] = value;
                G[j][i] = value;
            }
        }
    }

    printf("\nМатрица смежности графа G:\n");
    for(i = 0; i < N; i++) {
        printf("%3d", i);
    }
    printf("\n");

    for(i = 0; i < N; i++) {
        printf("%3d ", i);
        for(j = 0; j < N; j++) {
            printf("%3d", G[i][j]);
        }
    }
}
```

```

    printf("\n");
}

for(i = 0; i < N; i++) {
    loop[i] = G[i][i];
    for(j = 0; j < N; j++) {
        if(i != j) {
            deg[i] += G[i][j];
        }
    }

    deg[i] += 2*loop[i];
}

size = 0;
for(i = 0; i < N; i++) {
    for(j = i; j < N; j++) {
        if(G[i][j] == 1) {
            size++;
        }
    }
}

printf("\nРазмер графа G (количество ребер): %d\n", size);
printf("Количество вершин: %d\n", N);

printf("\nСтепени вершин:\n");
for(i = 0; i < N; i++) {
    printf("Вершина %d: степень %d", i, deg[i]);
    if(loop[i] > 0) {
        printf(" (включая %d петлю)", loop[i]);
    }
    printf("\n");
}

printf("\nСпециальные вершины:\n");

printf("Изолированные вершины (степень = 0): ");
int found_isolated = 0;
for(i = 0; i < N; i++) {
    if(deg[i] == 0) {
        if(found_isolated > 0) printf(", ");
        printf("%d", i);
        found_isolated++;
    }
}
if(found_isolated == 0) printf("отсутствуют");
printf("\n");

printf("Концевые вершины (степень = 1): ");
int found_end = 0;
for(i = 0; i < N; i++) {
    if(deg[i] == 1) {
        if(found_end > 0) printf(", ");
        printf("%d", i);
        found_end++;
    }
}
if(found_end == 0) printf("отсутствуют");
printf("\n");

printf("Доминирующие вершины (степень = %d): ", N-1);
int found_dominating = 0;
for(i = 0; i < N; i++) {
    if(deg[i] == N - 1) {

```

```

        if(found_dominating > 0) printf(" ");
        printf("%d", i);
        found_dominating++;
    }
}
if(found_dominating == 0) printf("отсутствуют");
printf("\n");

printf("\nДополнительная информация:\n");
int total_degree = 0;
for(i = 0; i < N; i++) {
    total_degree += deg[i];
}
printf("Сумма степеней всех вершин: %d\n", total_degree);
printf("Проверка (должна быть равна 2 * размер графа): %d\n", 2 * size);

for(i = 0; i < N; i++) {
    free(G[i]);
}
free(G);
free(loop);
free(deg);

_getch();
return 0;
}

```

Результат программы:

```

E:\users\Z3vvy1\B4\Debug\B4.exe
Введите количество вершин графа: 5
Матрица смежности графа G:
  0  1  2  3  4
0  1  1  0  1  1
1  1  0  0  0  0
2  0  0  1  1  0
3  1  0  1  1  0
4  1  0  0  0  1
Размер графа G (количество ребер): 8
Количество вершин: 5
Степени вершин:
Вершина 0: степень 5 (включая 1 петлю)
Вершина 1: степень 1
Вершина 2: степень 3 (включая 1 петлю)
Вершина 3: степень 4 (включая 1 петлю)
Вершина 4: степень 3 (включая 1 петлю)
Специальные вершины:
Изолированные вершины (степень = 0): отсутствуют
Концевые вершины (степень = 1): 1
Доминирующие вершины (степень = 4): 3
Дополнительная информация:
Сумма степеней всех вершин: 16
Проверка (должна быть равна 2 * размер графа): 16

```

Вывод:

Сгенерировать матрицу смежности для неориентированных графов G, вывести матрицу на экран, а также определить размер графа G, используя матрицу смежности графа и найти изолированные, концевые и доминирующие вершины.

